

Competitive Programming

Section 29 Editorial: Modular Arithmetic

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial map	4
2	A. Power Queries	5
2.1	Restatement	5
2.2	Constraints and traps	5
2.3	Observation and complete trace	5
2.4	Algorithm	5
2.5	Correctness proof	6
2.6	Complexity	6
2.7	C++ implementation notes	6
2.8	Python implementation notes	6
2.9	Full C++ solution	6
2.10	Full Python solution	7
2.11	Common wrong answers and tests	7
3	B. Fraction Queries	9
3.1	Restatement	9
3.2	Constraints and traps	9
3.3	Observation and complete trace	9
3.4	Algorithm	9
3.5	Correctness proof	9
3.6	Complexity	10
3.7	C++ implementation notes	10
3.8	Python implementation notes	10
3.9	Full C++ solution	10
3.10	Full Python solution	11
3.11	Common wrong answers and tests	11
4	C. Affine Repeater	12
4.1	Restatement	12
4.2	Constraints and traps	12
4.3	Observation and complete trace	12
4.4	Algorithm	12
4.5	Correctness proof	13
4.6	Complexity	13
4.7	C++ implementation notes	13
4.8	Python implementation notes	13
4.9	Full C++ solution	13
4.10	Full Python solution	14
4.11	Common wrong answers and tests	15
5	Testing discussion	16
6	What to carry forward	17

7	D. Nested Power Queries	18
7.1	Restatement and observation	18
7.2	Constraints and traps	18
7.3	Worked trace	18
7.4	Algorithm	19
7.5	Correctness proof	19
7.6	Complexity	19
7.7	Full C++ solution	19
7.8	C++ implementation notes	20
7.9	Full Python solution	20
7.10	Python implementation notes	20
7.11	Traps and tests	20
8	Stress testing	21
9	What to carry forward	22

1 Editorial map

Problem	Recognition signal	Core invariant	Time per query
A. Power Queries	exponent up to 10^{18}	<code>result*base^e</code> preserves original power	$O(\log b)$
B. Fraction Queries	stated prime modulus and quotient	denominator times inverse is one	$O(\log p)$
C. Affine Repeater	repeat one linear update	accumulated constant is a geometric sum	$O(\log k)$

Read the condition that makes an inverse legal before writing `MOD-2`. That condition, not the syntax of `pow`, is the difference between B and a general modular-division problem.

2 A. Power Queries

2.1 Restatement

For every query, output $a^b \bmod m$. Values satisfy $0 \leq a \leq 10^{18}$, $0 \leq b \leq 10^{18}$, and $1 \leq m \leq 10^9+7$. The result must be in $0..m-1$.

2.2 Constraints and traps

Repeated multiplication takes $O(b)$ time, which is impossible for the largest exponent. The modulus can be one, so the answer is zero even when $b=0$; the initial value must be $1 \% m$, not unconditional 1. In a reusable version with a larger modulus, multiplication also needs an overflow guard.

2.3 Observation and complete trace

The binary digits of 13 are 1101, so $3^{13}=3^8*3^4*3$. Binary exponentiation maintains the invariant below while processing those bits.

```
Compute 313 mod 17.
result  base  remaining exponent  action
1       3      13              take base: result=3
3       9       6          square and halve
3      13       3      take base: result=5; square and halve
5      16       1      take base: result=12? no: 5*16=80=12;
                        square and halve gives base=1, exponent=0
12      1       0      answer 12
```

The last line is a useful correction to mental arithmetic: $3^{13} \bmod 17$ is 12, not 5. Verify independently: $3^4=13$, $3^8=16$, and $3^{13}=16*13*3=624=12 \pmod{17}$.

2.4 Algorithm

```
pow_mod(a, b, m):
    result = 1 mod m
    base = a mod m
    while b > 0:
        if b is odd:
            result = result * base mod m
        base = base * base mod m
        b = b // 2
```

```
b = b // 2
return result
```

2.5 Correctness proof

Let A and B be the original base and exponent. At loop entry, $\text{result} * \text{base}^b == A^B \pmod m$. If b is odd, multiplying result by base and decreasing the represented exponent by one preserves that product. The remaining exponent is then even, so replacing base^b with $(\text{base}^2)^{(b/2)}$ also preserves it. When the loop ends, $b=0$, hence the invariant gives $\text{result} == A^B \pmod m$, exactly the requested answer.

2.6 Complexity

Each iteration halves b , so time is $O(\log b)$ and extra memory is $O(1)$. Across q queries this is $O(q \log b)$.

2.7 C++ implementation notes

Start with $1 \% \text{mod}$ for $\text{mod}=1$. The local bound permits either `long long` or `__int128` products; the listing uses `__int128` so the helper remains safe if the modulus is widened. Normalize a negative base in a more general input variant.

2.8 Python implementation notes

The built-in `pow(a,b,m)` is binary exponentiation, not an ordinary large power. The explicit loop below is included because it exposes the invariant; either form is suitable for the local bounds.

2.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

long long mod_pow(long long a, long long b, long long mod) {
    a %= mod;
    long long result = 1 % mod;
    while (b > 0) {
        if (b & 1LL) result = (long long)((__int128)result * a % mod);
        a = (long long)((__int128)a * a % mod);
        b >>= 1LL;
    }
    return result;
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    if (!(cin >> q)) return 0;
    while (q--) {
        long long a, b, mod;
        cin >> a >> b >> mod;
        cout << mod_pow(a, b, mod) << '\n';
    }
}

```

2.10 Full Python solution

```

import sys

def mod_pow(a: int, b: int, mod: int) -> int:
    result = 1 % mod
    a %= mod
    while b:
        if b & 1:
            result = result * a % mod
        a = a * a % mod
        b >>= 1
    return result

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    out = []
    for i in range(q):
        a, b, mod = data[1 + 3 * i : 4 + 3 * i]
        out.append(str(mod_pow(a, b, mod)))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

2.11 Common wrong answers and tests

Wrong approach	Why it fails	Essential test
loop b times result=1 use int products forget to reduce base	exponential-sized input range does not reduce modulo one squared residues can overflow unnecessary overflow and wrong negative handling	3 1000000000000000000 17 7 0 1 -> 0 a high-modulus adaptation a much larger than m

3 B. Fraction Queries

3.1 Restatement

The first line gives a prime p ($2 \leq p \leq 10^9+7$) and up to 200,000 queries. Every query gives a, b with $b \bmod p \neq 0$; output the residue for $a/b \bmod p$.

3.2 Constraints and traps

There is no ordinary division operator in a finite field. Floating point loses the exact residue, and integer division answers a different question. Fermat's inverse is legal only because p is prime and the denominator is nonzero modulo p . A composite-modulus version belongs to Section 30.

3.3 Observation and complete trace

An inverse inv satisfies $b * inv == 1 \pmod{p}$. For $10/4 \bmod 17$, use Fermat: $inv = 4^{(17-2)} = 4^{15} = 13 \pmod{17}$.

```
4 * 13 = 52 = 1 mod 17
10 * 13 = 130 = 11 mod 17
check: 4 * 11 = 44 = 10 mod 17
```

The check is a better debugging tool than converting any value to a decimal.

3.4 Algorithm

```
for each (a, b):
    inverse = pow_mod(b, p-2, p)
    print (a mod p) * inverse mod p
```

3.5 Correctness proof

Because p is prime and b is not divisible by p , Fermat's little theorem gives $b^{(p-1)} == 1 \pmod{p}$. Thus $b * b^{(p-2)} == 1$, so $b^{(p-2)}$ is the inverse of b . Multiplying a by this inverse is exactly the unique residue whose product with b is a ; therefore the algorithm prints $a/b \bmod p$.

3.6 Complexity

One inverse costs $O(\log p)$ by binary exponentiation. Total time is $O(q \log p)$ and extra memory is $O(1)$ apart from buffered I/O.

3.7 C++ implementation notes

Use the same `mod_pow` helper as A and cast the final product to `__int128` if you keep the generic modulus helper. Reducing `a` and `b` before the call makes the intent clear. Do not replace the helper by `a / b`.

3.8 Python implementation notes

`pow(b % p, p-2, p)` returns the inverse directly. Python's `/` creates a float and `//` is integer division; neither has a role in this solution.

3.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

long long mod_pow(long long a, long long b, long long mod) {
    long long result = 1 % mod;
    a %= mod;
    while (b) {
        if (b & 1LL) result = (long long)((__int128)result * a % mod);
        a = (long long)((__int128)a * a % mod);
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long p;
    int q;
    if (!(cin >> p >> q)) return 0;
    while (q--) {
        long long a, b;
        cin >> a >> b;
        long long inv = mod_pow(b, p - 2, p);
        cout << (long long)((__int128)(a % p) * inv % p) << '\n';
    }
}
```

3.10 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    p, q = data[0], data[1]
    out = []
    for i in range(q):
        a, b = data[2 + 2 * i : 4 + 2 * i]
        inv = pow(b % p, p - 2, p)
        out.append(str((a % p) * inv % p))
    print("\n".join(out))

if __name__ == "__main__":
    main()
```

3.11 Common wrong answers and tests

Wrong approach	Why it fails	Essential test
use / or a decimal	not finite-field division	p=17, 10 4 -> 11
use Fermat for arbitrary m	primality is required	compare with Section 30's composite case
forget $b \bmod p \neq 0$	inverse would not exist	p=2, b=1 is the boundary legal case
compute b^p normally	enormous intermediate value	large prime modulus

4 C. Affine Repeater

4.1 Restatement

For each of up to 200,000 queries, apply $f(x) = a \cdot x + b$ exactly k times modulo $MOD = 1,000,000,007$, where every input value and k may be as large as 10^{18} . Output $f^k(x)$.

4.2 Constraints and traps

Simulation is $O(k)$ and fails for large k . Inputs must be reduced before testing $a == 1$: $a = MOD + 1$ takes the same branch as one. There are two special cases: $k = 0$ returns the original x , and $a == 1$ makes the usual geometric division invalid.

4.3 Observation and complete trace

Unrolling gives

$$f^k(x) = a^k \cdot x + b \cdot (1 + a + \dots + a^{k-1}).$$

For the special branch, take $a=1, b=3, x=5, k=4$ modulo 17:

```
simulation: 5 -> 8 -> 11 -> 14 -> 0
formula:    1^4*5 + 3*(1+1+1+1) = 5+12 = 17 = 0 mod 17
```

When $a \neq 1$, the sum is $(a^k - 1) \cdot \text{inverse}(a - 1)$. When $a == 1$, it is k ; division by $a - 1$ would be division by zero.

4.4 Algorithm

```
reduce a, b, x modulo MOD
if k == 0: print x
ak = pow_mod(a, k, MOD)
if a == 1:
    geometric_sum = k mod MOD
else:
    geometric_sum = (ak - 1) * pow_mod(a - 1, MOD - 2, MOD) mod MOD
print (ak * x + b * geometric_sum) mod MOD
```

4.5 Correctness proof

For $k=0$, no update is performed and the formula gives x . Assume the formula after k updates. Applying $a*y+b$ one more time multiplies the old a^k*x term by a and appends one b to the existing sum, yielding the formula for $k+1$. Thus induction proves the affine expansion. The geometric sum identity is valid for $a \neq 1$; for $a=1$, all k summands equal one. The algorithm evaluates these exact quantities modulo the prime, so it returns $f^k(x)$.

4.6 Complexity

Two modular powers take $O(\log k + \log \text{MOD})$, which is $O(\log k)$ for the local fixed modulus; memory is $O(1)$ per query. Total time is $O(q \log k)$.

4.7 C++ implementation notes

Normalize all three coefficients before the branch. Write $(a^k - 1 + \text{MOD}) \% \text{MOD}$ before multiplying by the inverse to avoid a negative remainder. Products of residues fit in `long long` for this fixed MOD, but `__int128` is appropriate in a generic widened-modulus helper.

4.8 Python implementation notes

Use `pow(a,k,MOD)` and `pow(a-1,MOD-2,MOD)`. Python normalizes the subtraction after `% MOD`, but keeping $(a^k-1) \% \text{MOD}$ in the expression makes the intended residue explicit.

4.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    a %= MOD;
    if (a < 0) a += MOD;
    long long result = 1;
    while (b) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
```

```

cin.tie(nullptr);
int q;
if (!(cin >> q)) return 0;
while (q--) {
    long long a, b, x, k;
    cin >> a >> b >> x >> k;
    a = (a % MOD + MOD) % MOD;
    b = (b % MOD + MOD) % MOD;
    x = (x % MOD + MOD) % MOD;
    if (k == 0) {
        cout << x << '\n';
        continue;
    }
    long long ak = mod_pow(a, k);
    long long sum = (a == 1)
        ? k % MOD
        : (ak - 1 + MOD) % MOD * mod_pow(a - 1, MOD - 2) % MOD;
    cout << (ak * x + b * sum) % MOD << '\n';
}
}

```

4.10 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    out = []
    for i in range(q):
        a, b, x, k = data[1 + 4 * i : 5 + 4 * i]
        a %= MOD
        b %= MOD
        x %= MOD
        if k == 0:
            out.append(str(x))
            continue
        ak = pow(a, k, MOD)
        if a == 1:
            geometric_sum = k % MOD
        else:
            geometric_sum = (ak - 1) * pow(a - 1, MOD - 2, MOD) % MOD
        out.append(str((ak * x + b * geometric_sum) % MOD))

```

```

print("\n".join(out))

if __name__ == "__main__":
    main()

```

4.11 Common wrong answers and tests

Wrong approach	Why it fails	Essential test
simulate each update	k reaches 10^{18}	huge k with small values
use geometric quotient for a==1	inverse of zero does not exist	1 3 5 4
test original, not reduced, a omit k=0	MOD+1 is residue one formula's empty sum needs care	a=10000000008 any k=0 query
fail to normalize C++ subtraction	negative residue contaminates answer	a=0, k>0

5 Testing discussion

Run the supplied suite:

```
CP_TARGET=solution python3 sections/29_modular_arithmetic/check.py
```

For A, use a slow repeated product for exponents at most 30. For B, check that `b*answer mod p == a mod p`. For C, use a slow loop for `k<=20` and compare it with the formula on random reduced and unreduced inputs. The strongest fixed cases are modulus one, zero exponent, `p=2`, `a=0`, `a=1`, and `a=MOD+1`.

6 What to carry forward

Binary exponentiation is an invariant-preserving bit scan.
Modular division means multiplying by an inverse, never ordinary division.
Fermat inverses require a prime modulus and a nonzero denominator.
Repeated affine maps produce a power and a geometric sum.
Special cases such as $\text{mod}=1$, $\text{exponent}=0$, and $a=1$ belong in the proof first.

7 D. Nested Power Queries

7.1 Restatement and observation

We need $a^{(b^c) \bmod P}$, where $P = 1000000007$ is prime and a is nonzero modulo P . Fermat's theorem gives:

$$a^{(P-1)} = 1 \bmod P$$

so only the exponent modulo $P-1$ matters. Compute the inner exponent with a second binary exponentiation:

```
e = b^c mod (P-1)
answer = a^e mod P
```

For example, $2^{(3^2)} = 2^9 = 512$.

7.2 Constraints and traps

There can be up to 200000 queries, while b and c may be enormous. The outer base is guaranteed not to be divisible by P ; that contract is vital.

```
Never form b^c as an ordinary integer.
Reducing the exponent modulo P-1 is valid here only because gcd(a, P) = 1.
The exponent residue may be zero; pow(a, 0, P) correctly returns 1.
Use 64-bit multiplication in C++: products stay below about 10^18.
```

If the base could be zero modulo P , the exponent reduction would need a separate zero/nonzero analysis. This problem deliberately avoids that case.

7.3 Worked trace

Consider $a=2$, $b=3$, $c=5$. First compute the inner exponent modulo $P-1 = 1000000006$ by binary exponentiation:

$$e = 3^5 \bmod (P-1) = 243$$

Fermat permits replacing the true exponent 3^5 with this residue, so the second binary power computes $2^{243} \bmod P$. The two powers use the same tool; only their moduli differ. For a huge c , the first call still examines only the bits of c .

7.4 Algorithm

```
for each query a,b,c:
    exponent = power(b, c, P-1)
    print power(a, exponent, P)
```

Both calls are standard binary exponentiation.

7.5 Correctness proof

Lemma. `power(x,y,m)` returns $x^y \bmod m$.

Proof. Binary exponentiation maintains `result * baseremaining` equal to the original power modulo `m`; each bit either multiplies `result` by the current power of two or skips it. When `remaining` becomes zero, the invariant gives the result. $\square$

Theorem. The algorithm prints $a^{(b^c) \bmod P}$.

Proof. The first power call returns $e = b^c \bmod (P-1)$ by the lemma. Because `a` is nonzero modulo prime `P`, Fermat implies exponents differing by `P-1` give the same residue, so $a^{(b^c)} = a^e \bmod P$. The second power call returns that value by the lemma. $\square$

7.6 Complexity

Each query uses two logarithmic powers:

```
Time:  $O(q (\log b + \log c))$ 
Memory:  $O(1)$  besides output
```

7.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long power(long long base, long long exponent, long long mod) {
    long long result = 1 % mod;
    for (base %= mod; exponent; exponent >>= 1, base = base * base % mod)
        if (exponent & 1) result = result * base % mod;
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q; cin >> q;
```

```

while (q--) {
    long long a, b, c; cin >> a >> b >> c;
    cout << power(a, power(b, c, MOD - 1), MOD) << '\\n';
}
}

```

7.8 C++ implementation notes

Use `long long` for the products in binary exponentiation: both operands are less than P , and their product fits in signed 64-bit range. Normalize an input base with `% MOD` if the statement later allows values outside the usual range.

7.9 Full Python solution

```

import sys
MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    out = []
    for i in range(data[0]):
        a, b, c = data[1 + 3 * i:4 + 3 * i]
        out.append(str(pow(a, pow(b, c, MOD - 1), MOD)))
    print("\\n".join(out))

if __name__ == "__main__":
    main()

```

7.10 Python implementation notes

Python's three-argument `pow(base, exponent, modulus)` is binary exponentiation implemented in C. It avoids constructing the enormous integer `b ** c`; use it for both nested powers.

7.11 Traps and tests

- Reducing the outer exponent by $P-1$ requires a nonzero modulo P .
- Do not compute b^c as an ordinary integer.
- Test $b=1$, huge c , and base $P-1$ with odd/even exponent.

8 Stress testing

For tiny bases and exponents, compute $b**c$ as an ordinary integer and then multiply a that many times modulo P . This independent slow oracle validates the two modular power calls. The generator also emits 200000 queries with known answer 2 to exercise input and logarithmic query processing.

9 What to carry forward

Under a prime modulus, a nonzero base has exponent period dividing $P-1$. Nested powers are handled from inside out, reducing each exponent at the modulus required by the next power.

““